



Security Audit

Lend xyz (RWA)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Lend xyz team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Lend.xyz is a Web3 platform that enables users to invest crypto into tokenized real-estate projects. It tokenizes real-world real-estate assets (e.g. rental properties or commercial real-estate) into blockchain-native tokens, letting investors join with relatively small amounts (starting from 1 USDC / low barrier) and earn yields (reported around 8–15 % APR) via rents or property income. The platform aims to bridge traditional real-estate investment opportunities with DeFi liquidity and accessibility — offering fractional ownership, regulatory compliance (via a partner under MiCA/European regulation), and enabling on-chain trading or leverage of the tokenized real-estate assets.

Project Name: Lend xyz

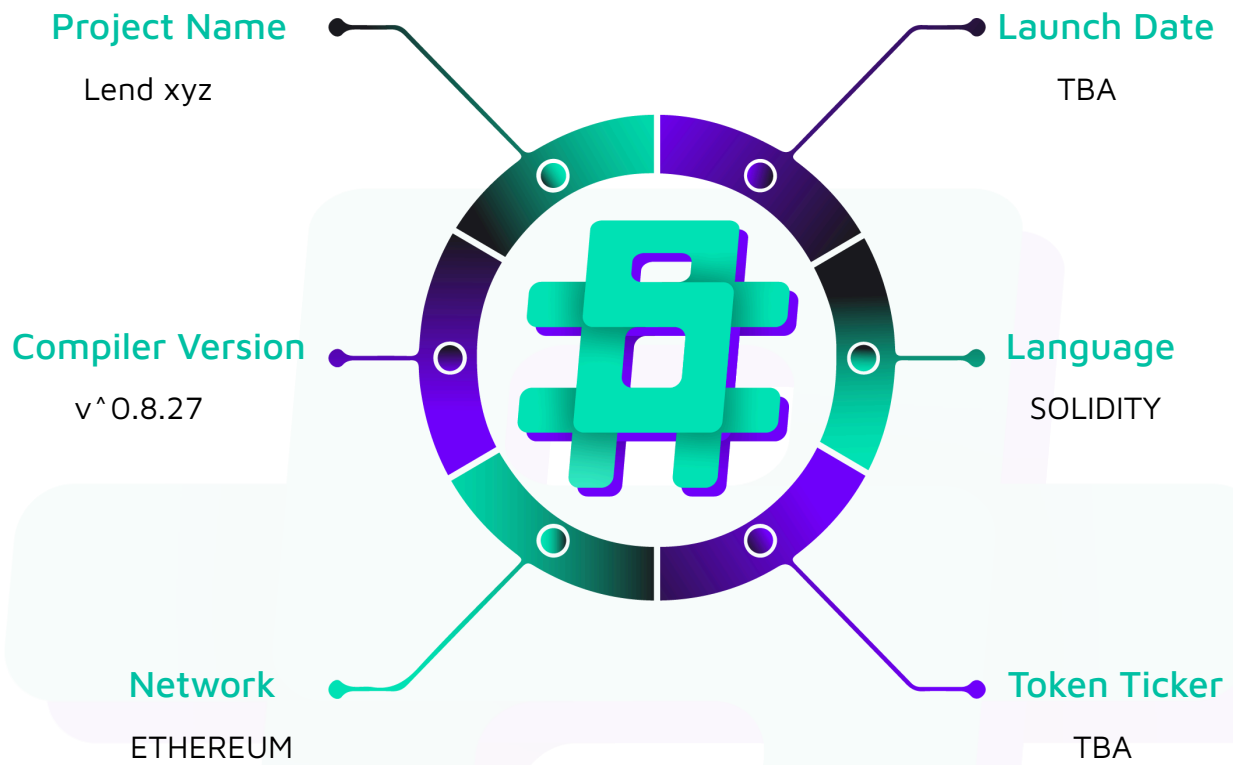
Project Type: RWA

Compiler Version: ^0.8.27

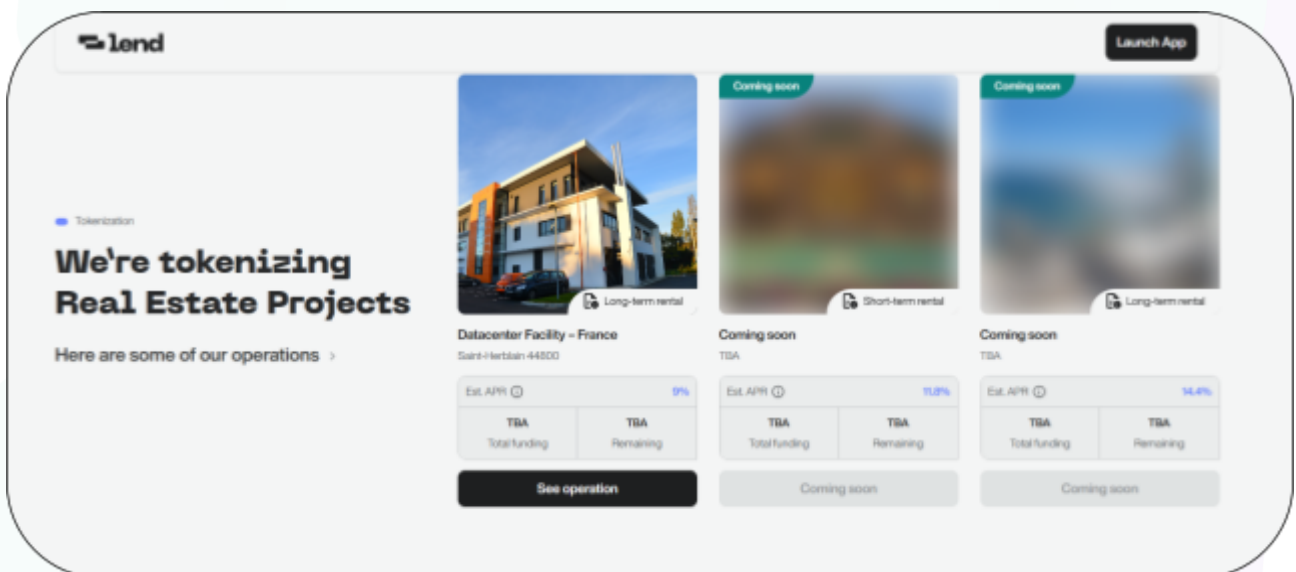
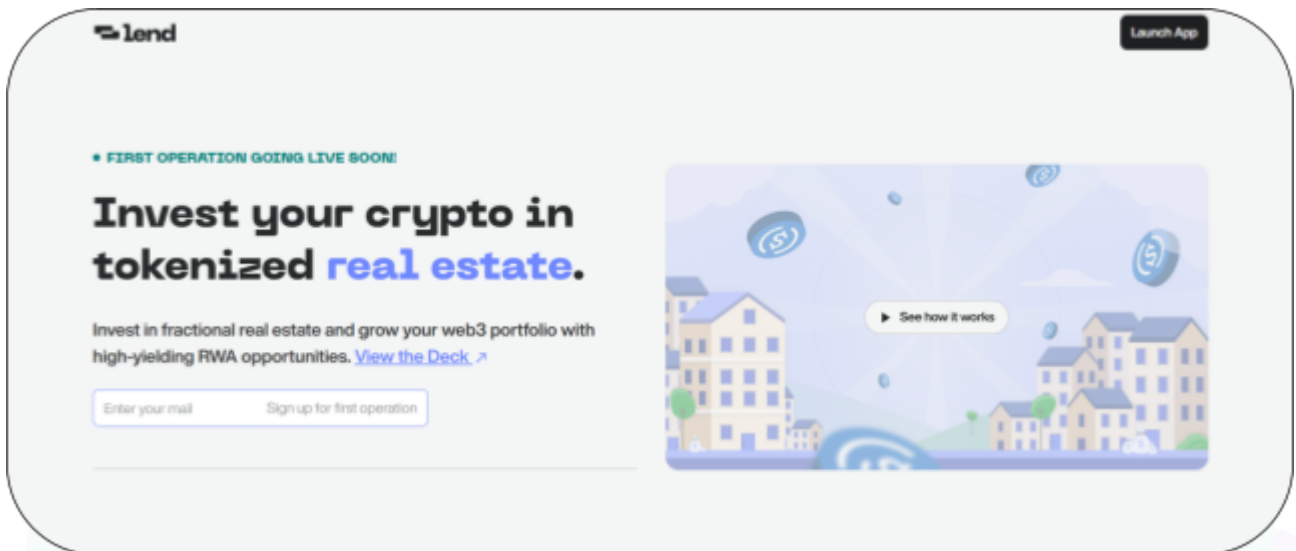
Website: <https://lend.xyz/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Lend xyz project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Lend xyz Smart Contracts
Platform	Ethereum / Solidity
Audit Date	December, 2025
Contract 1	Admin.sol
Contract 2	Invest.sol
Contract 3	Operations.sol
Contract 4	Utils.sol
Contract 5	Storage.sol
Audited GitHub Commit Hash	b26a4d18af87c0033159f6ee3748ebf5cc4a4639
Fix Review GitHub Commit Hash	ae227c34211fe9f9d92ec11b2c3eba820ee1c51d

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

1 Medium severity vulnerabilities

2 Low severity vulnerabilities

2 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Admin.sol Allows users to: - None Allows admins to: - Refund user USDC and burn opLEND balance - Batch refund selected users for an operation - Update EUR/USD oracle address used for pricing - Update backend signer for investment signatures - Set LayerZero peer for opLEND tokens - Withdraw raised USDC to destination after finish - Toggle blacklist status for a participant address	Contract achieves this functionality.
Invest.sol Allows users to: - Invest in operation via signature paying USDC - Invest and immediately bridge opLEND cross-chain - Predeposit before start via signature and USDC - Claim gifted or predeposited opLEND after start - Claim and bridge gifted/predeposited opLEND - Batch claim opLEND for multiple users - Query USDC cost for a shares amount - Query shares amount for given USDC funds Allows admins to: - Gift opLEND to a user while paying USDC	Contract achieves this functionality.
Operations.sol	Contract achieves this

Allows users to: - View operation data by id - Check if operation is finished Allows admins to: - Create new operation deploying opLEND token - Cancel an operation - Start an operation - Pause or resume funding - Enable or disable predeposits	functionality.
opLend.sol Allows users to: - Whitelist self using backend signature nonce - Transfer tokens between whitelisted addresses only - Use transferFrom between whitelisted addresses - Bridge tokens through LayerZero OFT transfers Allows admins to: - Mint tokens up to capped supply - Burn user tokens via adminBurn - Update backend signer address - Batch set LayerZero peers for remotes	Contract achieves this functionality.
Utils.sol Allows users to: - Library/Interface — callable helpers only Allows admins to: - Library/Interface — callable helpers only	Contract achieves this functionality.
Storage.sol Allows users to: - Library/Interface — callable helpers only Allows admins to: - Library/Interface — callable helpers only	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Lend xyz project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Lend xyz project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Utils#recoverSigner - Missing Domain Separation in Signed Messages Enables Cross-Chain / Cross-Contract Replay

Description

The protocol relies on backend-signed messages to authorize sensitive actions such as minting, whitelisting, or bridging. Although each signature includes random nonce that is consumed on use, the signed hash is not bound to a specific chain or contract instance. As a result, the same valid signature can be replayed on a fork, a cloned deployment, or another chain where nonce state is independent. This violates the assumption that backend approvals are single-use and domain-specific.

Vulnerability Details

Backend signatures are verified on-chain by reconstructing a hash from request parameters such as the user address, amount, and a random nonce. Once a signature is used, the nonce is marked as consumed in a `usedNonces` mapping, preventing reuse on the same contract instance. However, the signed hash does not include `block.chainid` or `address(this)`, meaning it is not cryptographically bound to a specific execution domain. Because each deployment maintains its own `usedNonces` mapping, the same signature remains valid across different chains or cloned contracts. This allows an attacker to replay a previously authorized action in a different domain using the exact same signature and nonce.

```
function recoverSigner(bytes32 ethSignedMessageHash, bytes memory signature) internal
pure returns (address) {
    (bytes32 r, bytes32 s, uint8 v) = splitSignature(signature);
    return ecrecover(ethSignedMessageHash, v, r, s); // no v/s canonicity checks
}
```

Impact

An attacker can reuse a single backend authorization to execute the same privileged action multiple times across different chains, forks, or cloned deployments, leading to duplicate token minting, repeated whitelist inclusion, or repeated bridge approvals that were intended to be single-use, ultimately breaking protocol integrity and potentially resulting in unbounded asset issuance across domains.

Recommendation

Bind the signed message to its execution domain by including both `block.chainid` and `address(this)` in the signed hash. This ensures that a signature is only valid for one specific contract on one specific chain and cannot be replayed elsewhere.

Status

Resolved

Medium

[M-01] Admin#refundUser - Bridged opLEND Tokens Block Refunds

Description

Users who bridge their opLEND tokens to another chain have their local balance burned, but their USDC contribution remains recorded on the source chain. The refundUser flow depends on the on-chain token balance and burns the user's tokens before paying back USDC. After a bridge, the local balance is zero, causing refundUser to revert with NoOpLendBalance and making the refund path unusable for bridged participants.

Vulnerability Details

refundUser reads the current opLEND balance and reverts when it is zero, then burns that balance prior to transferring USDC. LayerZero bridging calls _debit, which burns the sender's balance on the source chain before minting on the destination chain, so any bridged user has a zero balance locally. As a result, the refund call reverts for bridged users even though their usdcRaisedPerClient entry is positive. There is no cross-chain check or requirement to bring tokens back before refunding, leaving bridged contributors unable to exit.

```
// src/facets/Admin.sol
uint256 opLendBalance = opToken.balanceOf(user);
...
if (opLendBalance == 0) revert Events.NoOpLendBalance();
...
opToken.adminBurn(user, opLendBalance);
require(s.usdc.transfer(user, userInvestAmount), Events.TransferFailed());
```

```
// src/opLend.sol
function _debit(address _from, uint256 _amountLd, uint256 _minAmountLd, uint32 _dstEid)
    internal
    override
    returns (uint256 amountSentLd, uint256 amountReceivedLd)
{
```

```
require(whitelisted[_from] == true, "User is not whitelisted");  
(amountSentLd, amountReceivedLd) = _debitView(_amountLd, _minAmountLd, _dstEid);  
_burn(_from, amountSentLd); // burn local balance on bridge out  
}
```

Impact

Bridged investors cannot be refunded on the source chain: any refund attempt reverts before transferring USDC, so their principal is permanently locked while the protocol's accounting still reflects the outstanding liability. This stalls remediation for cancellations or emergency refunds and leaves affected users' funds stuck without a recovery path.

Recommendation

Track and refund based on recorded contributions or share allocations rather than the current on-chain opLEND balance, and only require token burn if the balance actually exists on this chain. For example, store a non-transfer-adjusted `userShares[id][user]` when minting, use it for the refund decrement, and gate refunds to bridged users by requiring they bridge tokens back or by initiating a cross-chain burn before transferring USDC.

Status

Acknowledged (intentional centralized design for regulatory compliance)

Low

[L-01] Operations#setPredeposits - Redundant events emitted on unchanged state

Description

The function emits PredepositsOpen/PredepositsClosed events even when `predepositsOpen[id]` is already in the requested state. This produces misleading event logs that suggest a state transition when none occurred, increasing monitoring noise and potential misinterpretation by off-chain systems.

Recommendation

Guard the emit with a state change check to only log real transitions, e.g.:

```
if (s.predepositsOpen[id] != state) {  
  s.predepositsOpen[id] = state;  
  emit state ? Events.PredepositsOpen(id) : Events.PredepositsClosed(id);  
}
```

Status

Resolved

[L-02] Invest#_invest - Rigid totalShares Check Enables Front-Run Denial

Description

The purchase cap check reverts whenever `sharesAmount` pushes `fundingProgress` above `totalShares`, instead of filling the remaining capacity. If an attacker slightly front-runs near the cap, later buyers holding signatures for a higher share count cannot adjust downward and are reverted. Residual capacity remains unused while legitimate investors waste gas or must obtain new signatures.

Recommendation

Clamp the purchase to the remaining capacity or validate signatures against a maximum rather than an exact amount. Example fix: cap `sharesAmount` to the available remainder before cost/signature checks and proceed without reverting.

```
uint256 remaining = s.operations[id].totalShares - s.fundingProgress[id];  
if (remaining == 0) revert Events.OpFinished();  
if (sharesAmount > remaining) sharesAmount = remaining;
```

Status

Acknowledged (by design; exact share amounts are required due to regulatory constraints)

QA

[Q-01] Admin#updateOracleAddress / updateBackendSigner - Missing zero address validation

Description

Both `updateOracleAddress` and `updateBackendSigner` accept arbitrary inputs without rejecting the zero address, allowing the owner to accidentally set critical dependencies to `address(0)`. A zero oracle address breaks exchange rate retrieval, while a zero backend signer disables whitelist verification. The protocol can be left in an unusable state until another privileged transaction restores valid addresses.

Recommendation

Add explicit checks to ensure `newOracleAddress` and `newBackendSigner` are nonzero before assignment, reverting otherwise (e.g., `require(newOracleAddress != address(0), "zero address");`).

Status

Resolved

[Q-02] Admin#batchRefundUsers - Missing len validation can revert batch refunds

Description

`batchRefundUsers` iterates up to caller-supplied `len` without checking it against `users.length`. If `len` exceeds the array length, the function reverts mid-loop, aborting the entire batch. This prevents intended refunds and can block admin operations due to a single malformed parameter. Defensive validation is expected for batching helper functions.

Recommendation

Validate `len` against `users.length` before looping and consider rejecting zero-length batches. Example fix:

```
function batchRefundUsers(uint256 id, address[] calldata users, uint256 len) external {
    LibDiamond.enforceIsContractOwner();
    require(len > 0 && len <= users.length, "invalid len");
    for (uint256 i = 0; i < len; i++) {
        this.refundUser(id, users[i]);
    }
}
```

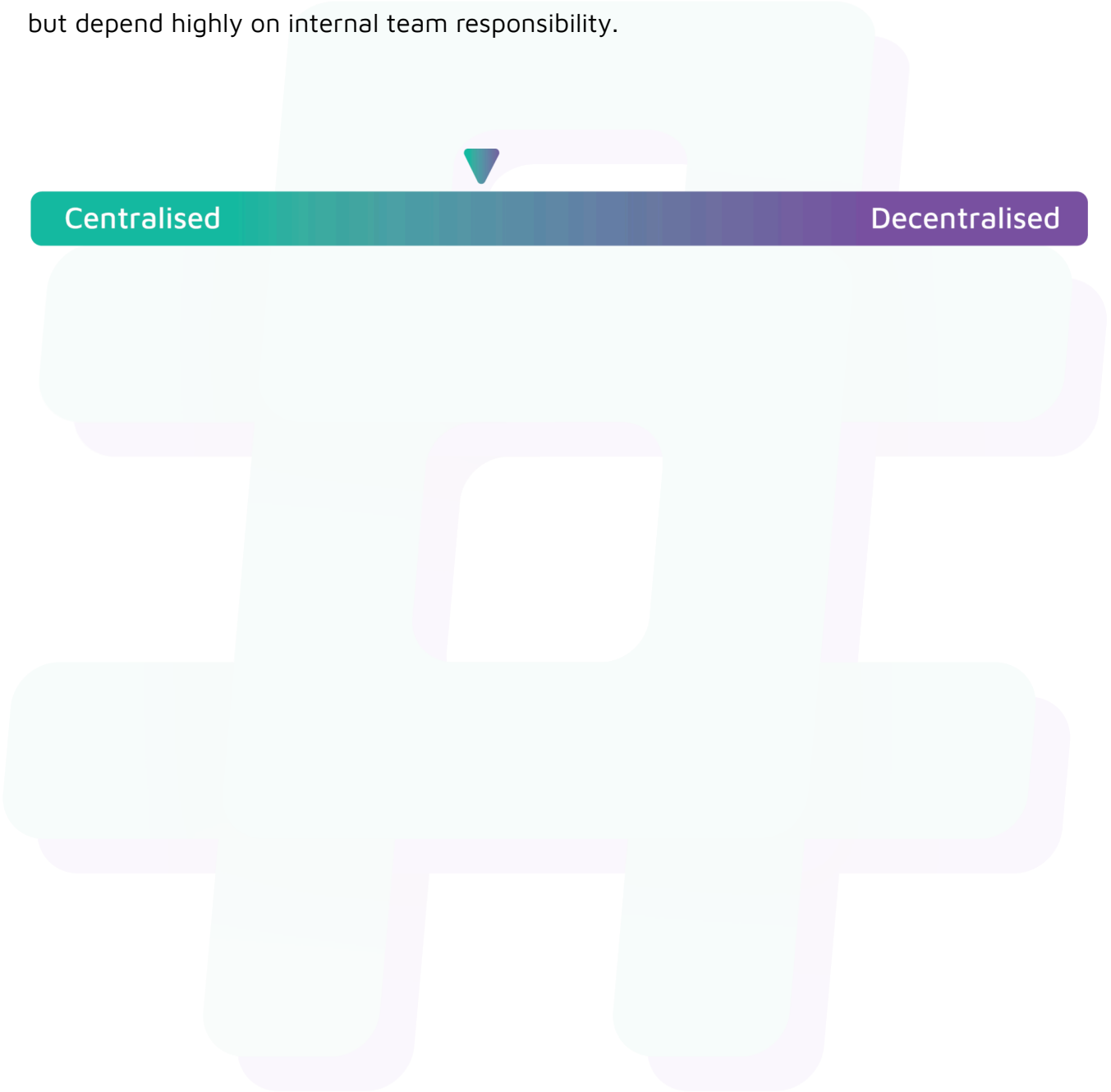
Status

Resolved

Centralisation

The Lend xyz project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Lend xyz project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.